Codebase Onboarding Agent — Project Report

Created as part of an assignment for Codecademy's Building Agentic AI Applications bootcamp, with Satyajit Pattnaik, Fall 2026.

```
ask("How does signing work, step by step?",
    repo_url="https://github.com/pallets/itsdangerous")
```

That call sums up the project. Starting on someone else's codebase is rough: lots of files, no idea where anything lives. So you point this tool at any public Python repo on GitHub and ask questions in plain English. RAG over a vector database finds the relevant code by meaning, and a LangGraph agent reads it and explains it, citing exact files and lines. You can run it as a Streamlit web app (`app.py`, new in this version) or as a Google Colab notebook with a Gradio page. Unfamiliar terms are in the glossary at the end.

Live app: codebase-app-agent-xafqjmzxpga43vvrsckbj7.streamlit.app · Code: github.com/jvs867/codebase-onboarding-agent

System architecture

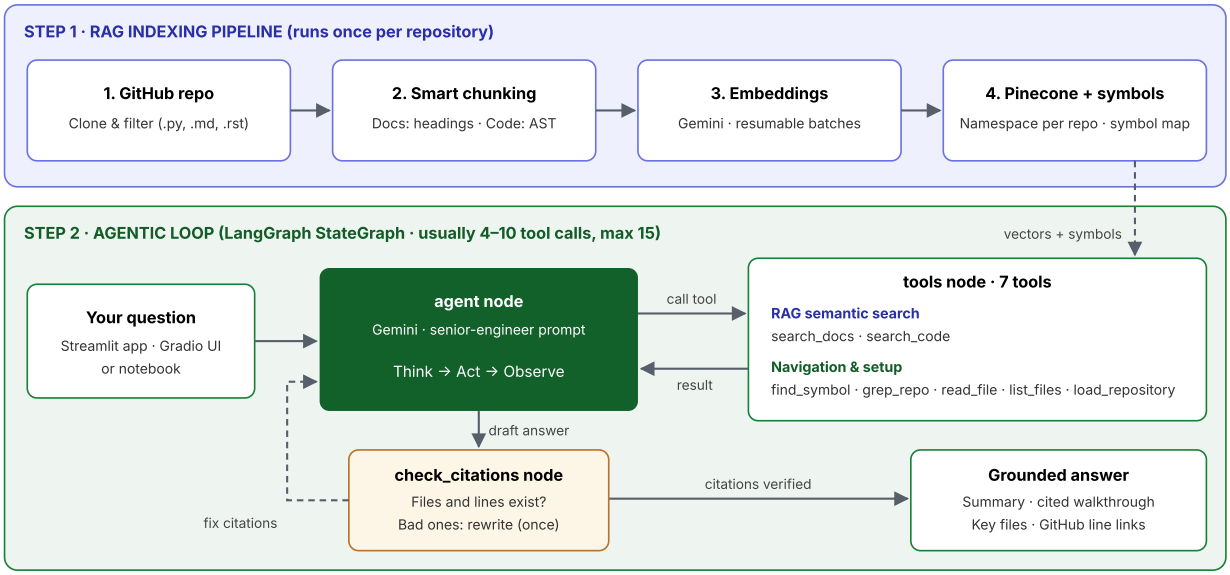


Figure 1: The whole system. Step 1 prepares a repo. Step 2 loops between the agent and tools nodes, then checks the citations before answering.

Step 1: The RAG pipeline

```
tree = ast.parse(source)
for node in tree.body:      # one chunk per function, class, method
    ...
```

That loop is the heart of indexing. The knowledge base here isn't a tidy stack of documents: it's a whole repo, docs and source code together, and its structure is worth keeping. The first time a repo is loaded, `prepare_repository()` gets it ready to be searched by meaning:

1. Download the repo (latest version only).
2. Filter: keep Python files and docs, skip tests, huge files and build folders.
3. Chunk by structure: docs are cut at headings (up to 2,000 characters). Code is cut with the `ast` module, as above, into one chunk per function, method, class and module (up to 6,000 characters), so a function never gets sliced in half. The same pass builds a lookup table of where everything is defined.
4. Embed: Gemini turns each chunk into an embedding. A rate limiter keeps the requests under the free plan's limit.
5. Store the embeddings in a Pinecone serverless index (cosine similarity), one namespace per repo, so they're still there next session.

> A repo that's already indexed skips steps 4–5, and an interrupted upload resumes with only the missing chunks.

Step 2: The agentic workflow

```
builder.add_edge(START, "agent")
builder.add_conditional_edges("agent", route_after_agent, ["tools", "check_citations"])
builder.add_edge("tools", "agent")
builder.add_conditional_edges("check_citations", route_after_check, ["agent", END])
```

Those four lines are the agent's entire control flow: a LangGraph `StateGraph` with three nodes. Basic RAG searches once and answers. Here the agent loops (think, pick a tool, look at the result) until it has enough evidence.

Node	What it does
`agent`	Gemini either asks for tool calls or writes the answer. Backup models step in on errors.
`tools`	Runs the tools it asked for and hands the results back to `agent`.
`check_citations`	Checks that every cited file and line range really exists. A bad answer goes back once with the problems listed, e.g. "*`signer.py` does not exist (did you mean `src/itsdangerous/signer.py`?)*"

`MAX_TOOL_CALLS = 15`: after that, the agent has to answer without tools. A checkpointer (`InMemorySaver`) remembers each conversation so follow-up questions work.

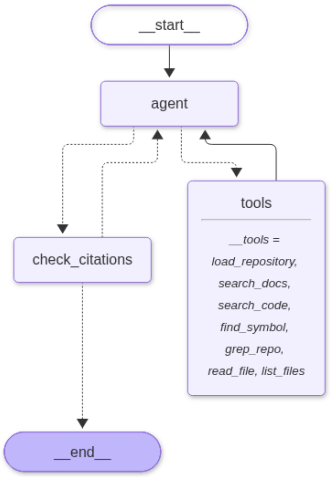


Figure 2: The agent graph as drawn by LangGraph. Dotted arrows are conditional.

The agent's 7 tools

Tool	Type	What it does
`load_repository`	Setup	Downloads and indexes a repo (Step 1).
`search_docs`, `search_code`	RAG	Search docs or code by meaning.
`find_symbol`	Navigation	Jumps to where a function or class is defined.
`grep_repo`	Navigation	Searches for exact text, like Ctrl+F.
`read_file`	Navigation	Reads only the lines it needs.
`list_files`	Navigation	Lists folders and files.

`SYSTEM_PROMPT` tells the agent to behave like a senior developer helping a new teammate: never guess, start with the docs, then trace the code. Every answer has a summary, numbered steps citing `file:START-END`, and the files to read first.

The Streamlit app

```
streamlit run app.py
```

One command, and the whole thing runs in your browser. The goal was a page you can use without reading instructions: paste a link, ask, and watch the agent work. The notebook's Gradio page only exists while the notebook is running, and its link is temporary. `app.py` packs the same pipeline, tools and agent into one file that also hosts for free on Streamlit Community Cloud, with a permanent link.

- Sidebar: load a repo (with a progress bar), start a new chat, re-index, and list or delete indexed repos.
- Chat: each tool call shows up as a step you can open, then the citation check, then the answer with GitHub links.
- Keys and login: API keys come from environment variables, Streamlit secrets or the sidebar. An optional username and password protect the app.

	Notebook (Colab + Gradio)	Streamlit app (`app.py`)
Where it runs	Google Colab	Your computer, or Streamlit Community Cloud
How to start it	Runtime → Run all	`streamlit run app.py` , or open the link
Web link	Temporary `gradio.live` link	Permanent `streamlit.app` link
Evaluation	Included (section 16)	Not included; same agent

```
@st.cache_resource
def get_backend(google_api_key, pinecone_api_key) -> Backend: ...
```

This decorator solves the one real design problem. Streamlit reruns the whole script every time you click or type, so ordinary variables vanish. Anything shared by everyone (connections, loaded repos, the agent) lives in one `Backend` object that `st.cache_resource` builds once. Each visitor's chat lives in `st.session_state` .

Results

The measured runs come from the notebook, on `pallets/itsdangerous` (9 docs files, 9 Python files, 88 symbols, 105 embeddings) with `gemini-3.1-flash-lite` .

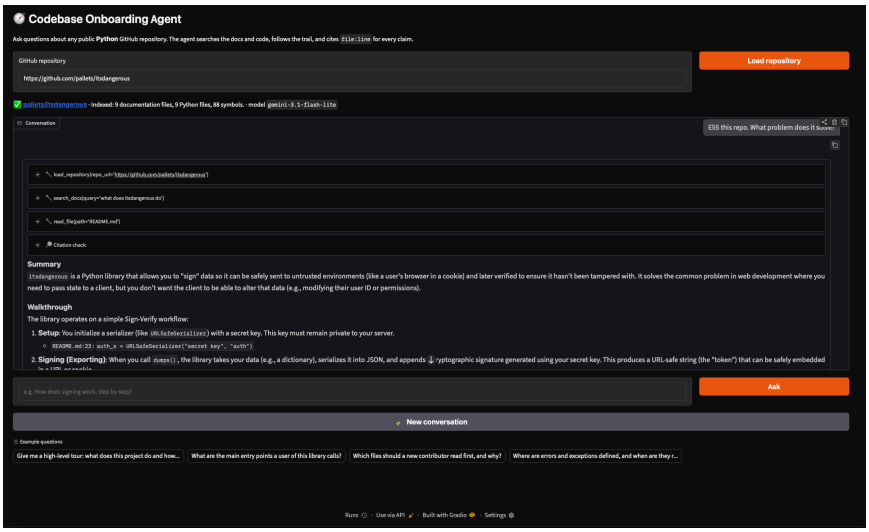


Figure 3: The notebook's Gradio UI. Tool calls and the citation check appear as steps above the answer. The Streamlit app presents the same steps.

Notebook: asked for a high-level tour, the agent first cited short filenames like `signer.py` . The citation check flagged all six and suggested full paths, and the rewrite passed with 11 verified citations in 9 seconds.

Streamlit app: it ran on a Mac with real keys and was then deployed to the live link above. Before that it was tested with stand-in versions of Gemini and Pinecone, which reproduced the notebook's file, symbol and embedding counts. No timings were measured.

Evaluation

```
df = run_eval()   # 8 questions, each in a fresh conversation
```

A demo that looks good isn't proof, so section 16 of the notebook tests the agent on `pallets/itsdangerous`. A question passes if the expected file is cited and every citation is valid. The last one is a trick question on purpose: it passes if the agent admits the library doesn't do it.

Question (shortened)	Must cite	Tool calls	Result
How Signer creates and checks a signature	`signer.py`	3	✅
Timestamps and max_age expiry	`timed.py`	4	✅
Exception for an expired signature	`exc.py`	3	✅
How Serializer.dumps signs an object	`serializer.py`	6	✅
How URLSafeSerializer makes output URL-safe	`url_safe.py` or `encoding.py`	4	✅
Signer's key derivation options and default	`signer.py`	3	✅
Where URL-safe base64 is implemented	`encoding.py`	2	✅
Saving tokens to PostgreSQL *(out of scope)*	Nothing: must say it isn't there	2	✅

> Result: 8/8 passed, with all 37 citations valid and an average of 3.4 tool calls and 5 seconds per question.

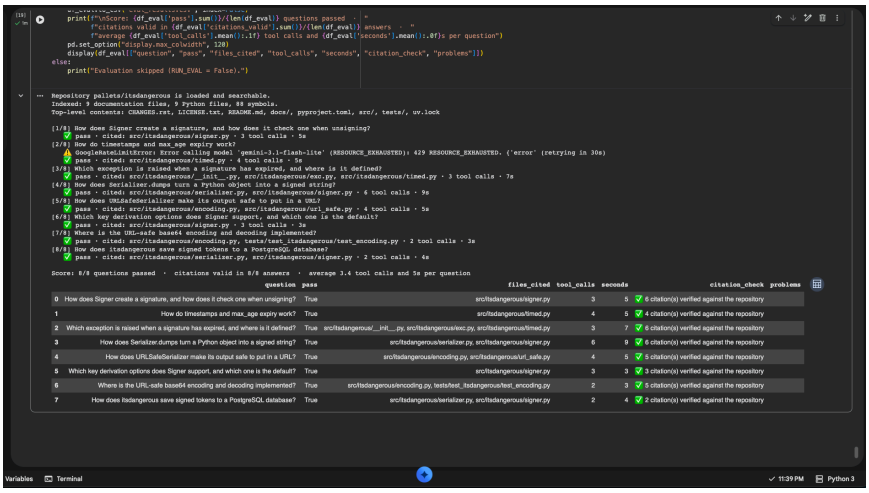


Figure 4: Evaluation output in the notebook.

Design challenges

Challenge	How it's handled
Gemini rate limits	`EmbeddingRateLimiter` paces embedding requests under the free plan's 30,000 tokens per minute. At start-up each chat model gets a 20-second test, and the ones that respond become the main model and its backups.
Interrupted indexing	`_chunk_id()` gives a chunk the same ID every time, so only missing chunks get embedded.
Chunking code	Fixed-size cuts would split functions. AST chunks keep each one whole.
Wrong citations	Models cite short filenames, missing files or lines past the end of a file. `check_citations` catches these and asks for one rewrite.
Several users	`with backend.lock:` All tools share one "active repository", so a second question waits for the first.
Secrets	Keys come from Colab Secrets, environment variables or Streamlit secrets, never from the code.
Python version	`langchain-pinecone` won't install on Python 3.14, so the project asks for 3.10–3.13.

Limitations & future work

- Python only: a parser such as tree-sitter could add other languages.
- No automatic refresh: a changed repo has to be re-indexed by hand.
- Search by meaning only: adding keyword search (hybrid search) could help with exact names.
- Small evaluation: eight questions on one repo, checking citations only, and not available in the Streamlit app.
- The hosted app forgets on restart: chats and downloaded repos are temporary. Embeddings stay in Pinecone.

How to use it

Hosted app: open the live link, paste a GitHub link in the sidebar, click Load repository, and ask away.

On your own computer (Python 3.10–3.13 and free API keys from Google AI Studio and Pinecone):

```
python3.12 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
export GOOGLE_API_KEY="..." PINECONE_API_KEY="..."
streamlit run app.py
```

In Colab: open the notebook, add the two keys under Secrets, then Runtime → Run all. To host your own copy, see the README.

Glossary

Term	Meaning
LLM	An AI model that reads and writes text. This project uses Google's Gemini.
Chunk	A small piece of a file, such as one function or one docs section.
Embedding	A list of numbers that represents a text's meaning. Similar meanings get similar numbers.
Vector database	Stores embeddings and finds the ones closest to a question. Here: Pinecone.
RAG	Retrieval-Augmented Generation: fetch the relevant chunks first, then let the LLM answer from them.
Agent	An LLM in a loop: pick a tool, look at the result, repeat until it can answer.
LangGraph	A library for building an agent as a graph of steps (nodes) and arrows (edges).
AST	Abstract Syntax Tree: Python's structured view of code, showing where each function and class starts and ends.